activate environment

python3 -m venv .venv source .venv/bin/activate

install the dependency

pip install -r requirements.txt

initialize project to connect with the db

python3 models.py

run models

python3 manage.py makemigrations forecasting python3 manage.py migrate

pipeline

python3 train_pipeline.py

run server

python3 manage.py runserver

DOCUMENTATION

Try testing these specific control scenarios: Scenario A (Worst Case Logistics Chaos): Go to this URL to see how much demand drops when there is bad weather and bad traffic: http://127.0.0.1:8000/?weather=1&traffic=1&holiday=0

Scenario B (Peak Holiday Surge): Go to this URL to check your stock risk during clear weather on a holiday: http://127.0.0.1:8000/?weather=0&traffic=0&holiday=1

How the Random Forest Model Makes Decisions To understand how you are controlling it, here is what the model does when you change those numbers:

Your views.py extracts the weather, traffic, and holiday values from your browser request.

It packages them into a numerical array structure that looks like this: [[weather, traffic, holiday]].

The script calls model.predict([[1, 0, 1]]) using your saved demand_rf_model.pkl binary.

The Random Forest combines the decisions of dozens of mini-decision trees built during your train_pipeline.py step and outputs the finalized forecast on your dashboard.